

IA utilizada: DeepSeek

Prompt: ¿Es conveniente usar métodos modificadores en los constructores? Justificación de al menos 1/2 cuartilla.

Respuesta: En general, no es conveniente usar métodos modificadores (setters) dentro de los constructores, aunque puede haber excepciones muy controladas. La razón principal es que un constructor debe inicializar un objeto de forma segura y consistente, mientras que un setter está pensado para modificar un objeto ya construido. Mezclar ambos usos suele introducir riesgos de diseño y errores difíciles de detectar.

Uno de los problemas más graves es el despacho dinámico de métodos. En lenguajes orientados a objetos como Java o C#, si el constructor llama a un setter público y este es sobrescrito por una subclase, se ejecutará la versión de la subclase antes de que sus propios campos estén inicializados. Esto puede provocar estados inconsistentes, excepciones o comportamientos inesperados. Por eso se recomienda no invocar métodos sobrescribibles desde un constructor.

Otro riesgo son los efectos secundarios. Un setter puede validar, registrar eventos, notificar observadores, actualizar cachés o persistir datos. Si se usa durante la construcción, el objeto aún no está completamente formado y podría exponerse una versión parcial o inválida de sí mismo. Además, si el setter lanza una excepción después de haber producido efectos externos, la construcción falla pero el sistema puede quedar alterado.

También está el tema de los invariantes. El constructor debe garantizar que el objeto nace cumpliendo todas sus reglas de negocio. Si se delega la asignación a setters que permiten validaciones más débiles o parciales, se corre el riesgo de crear objetos inconsistentes. Incluso si los setters validan, usarlos en el constructor acopla la creación del objeto a la API pública de modificación, lo que dificulta cambiar el diseño sin romper la construcción.

Ahora bien, hay casos donde podría ser aceptable: si los setters son privados o finales, no tienen efectos secundarios, solo asignan y validan, y la clase no está diseñada para herencia. Aun así, suele ser mejor usar métodos auxiliares privados o validar directamente en el constructor, evitando depender de la API de modificación. Para objetos complejos, patrones como el builder o las fábricas estáticas ofrecen una solución más limpia.

En conclusión, usar métodos modificadores en constructores no es una práctica recomendable por defecto. Aunque pueda reducir duplicación en casos muy específicos, los riesgos de herencia, efectos secundarios e inconsistencia superan la conveniencia. Lo más seguro es asignar y validar directamente en el constructor o mediante ayudantes privados, reservando los setters para la modificación posterior del objeto.